What the thesis concludes, and what it leaves behind

The evidence divides by what is being measured before it divides by resource

Safety: a difference in kind

Safe Rust rejects all 7 injected memory-safety and data-race classes at compile time. The 7 boundary classes, from a deadlock to a forgotten lock guard, compile and misbehave exactly as they do in C++.

Rust eliminates data races, not race conditions, and aliasing errors, not lock-ordering errors.

Migration: the call tax is bounded

An unmodified Rust core can be driven from C++ across a thin seam that holds every unsafe site. Any call tax is bounded at 10.91%.

API design and training time were not measured.

Performance: The order splits

Below saturation Rust leads.
Median gap at most 1.5 µs.
The p99.9 gap is larger.
The far tail is larger still.
A late wake-up sits in each.

The core, timed apart, is 1.6× faster in Rust.
Past saturation, the order follows what the timer includes.

What this adds to the decision is an instrument, not a verdict

- A frozen specification
- A parity-tested pair
- A reproducible framework
- A catalogue of what the guarantee misses

Where it goes next

large pages, production traces, async versus threads, two allocators, and review effort.

- Large pages
- Production traces
- Async versus threads
- Two allocators
- Review effort

Where the service still meets the sent rate, the choice is which defects to make impossible to write.